Fluxbase

Project Overview & Architecture

1. The Problem

Modern web and application development demands rapid iteration and zero-friction deployments. However, the database layer remains a significant bottleneck for fast-moving startups and independent developers. Traditional relational databases (PostgreSQL, MySQL) require complex infrastructure provisioning, continuous maintenance, connection pooling management, and complicated networking (VPCs, firewalls). Conversely, serverless NoSQL databases (Firestore, MongoDB) offer infinite scale and zero-maintenance but force developers to abandon the universal, powerful, and familiar syntax of SQL, leading to compromised data modeling and difficult migrations.

There is a critical gap between the Developer Experience (DX) of Serverless and the Power of Relational SQL.

2. Objective

Fluxbase aims to bridge this gap by providing a next-generation DBaaS (Database-as-a-Service) platform. The core objective is to offer developers the ability to spin up fully isolated, production-ready PostgreSQL or MySQL environments instantly via a beautiful UI dashboard, without touching an AWS console or managing a single connection string. Fluxbase handles the complex server-side connection pooling, tenant isolation, and SQL execution, exposing a simple, authenticated HTTP API for client applications to query their data securely from anywhere.

3. How the App Works

Fluxbase is a custom server-based SQL Engine built specifically for Next.js applications and Serverless architectures. Instead of forcing frontend or edge applications to manage heavy, long-lived TCP connections to a database, Fluxbase acts as an intelligent, stateless execution proxy.

- **Tenant Provisioning:** Through the Fluxbase dashboard, users create "Projects" and select their preferred SQL dialect (PostgreSQL or MySQL).
- **Schema Isolation:** Fluxbase dynamically provisions isolated schemas (search_path in PG, USE database in MySQL) within a highly-available, master AWS RDS instance.
- **API Layer:** Users generate scoped, secure API keys associated with their project.
- **Execution Engine:** Client applications send raw SQL strings over standard HTTPS to the Fluxbase /api/execute-sql endpoint.
- **Connection Pooling:** The Fluxbase server maintains aggressive, optimized connection pools to AWS RDS (utilizing pg and mysql2 drivers with TCP Keep-Alive). It authenticates the request, secures the isolated schema context, executes the SQL string natively on the bare-metal RDS engine, and returns standard JSON rows.

4. Cost Saving & Multi-Tenancy (Schema Tenancy)

A major advantage of Fluxbase is its extreme cost efficiency achieved through Multi-Tenant Schema Isolation on a Single Global Master Database.

Traditional DBaaS Providers: Typically spin up a dedicated Virtual Machine (VM) and Database Engine for each new project. This leads to massive overhead, paying for idle CPU/RAM resources across thousands of small projects.

The Fluxbase Approach: Fluxbase runs a single, highly-scaled Master Database natively on AWS RDS. When a new project is created, Fluxbase does not create a new machine. Instead, it creates a new logical Schema (or logical Database in MySQL) alongside dynamic, isolated roles.

The Result: Thousands of users can share the same underlying compute resources, drastically driving down the price per user to fractions of a cent, while maintaining 100% data boundary security. Connection pooling is managed centrally by the Fluxbase application, preventing AWS RDS from being overwhelmed by too many direct client connections.

5. Technology Stack

Frontend & UI

- **Framework:** Next.js 15 (App Router)
- **Language:** TypeScript
- **Styling:** Tailwind CSS
- **Component Library:** shadcn/ui & Radix UI
- **Code Editor:** Monaco Editor (React Monaco)
- **Data Visualization:** Recharts, React Flow (ERD Vis)

Backend & API

- **Framework:** Next.js Server Actions & Route Handlers
- **Authentication:** NextAuth.js (Google OAuth)
- **Validation:** Zod

Database Infrastructure

- **Primary State Store:** PostgreSQL (Master Instance) — Metadata, API Keys
- **Dialect Engine 1:** AWS RDS PostgreSQL 16+ (Tenant Data)
- **Dialect Engine 2:** AWS RDS MySQL 8.0+ (Tenant Data)
- **Drivers:** pg (PostgreSQL), mysql2/promise (MySQL)

6. Architecture & Flow Diagram

Request Lifecycle Flow

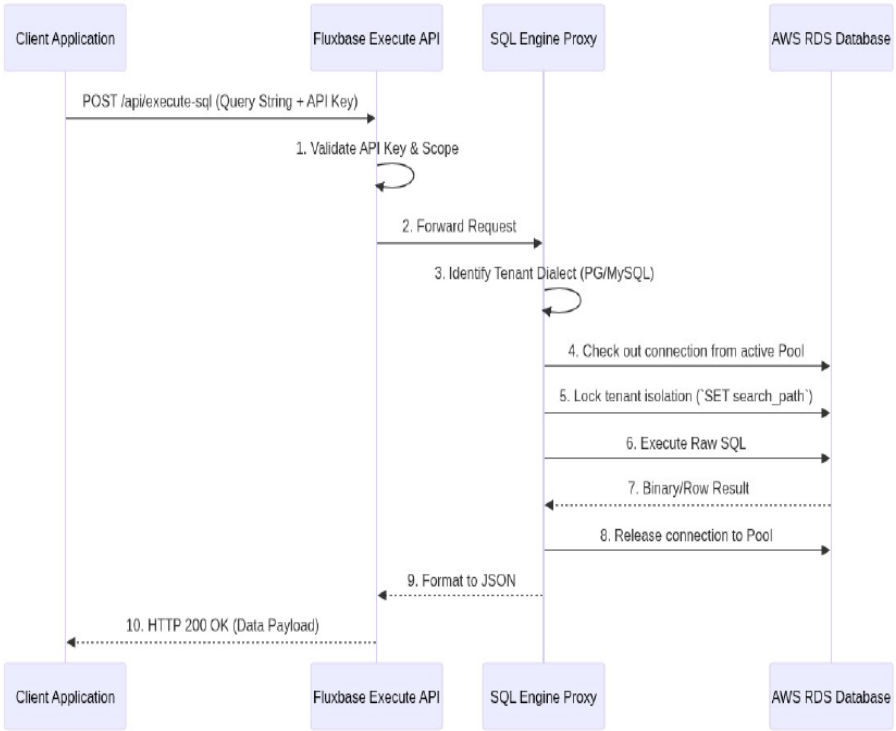

Figure 1 — Request Lifecycle: Client → Fluxbase Execute API → SQL Engine Proxy → AWS RDS

7. AWS Architecture Diagram

Infrastructure Topology

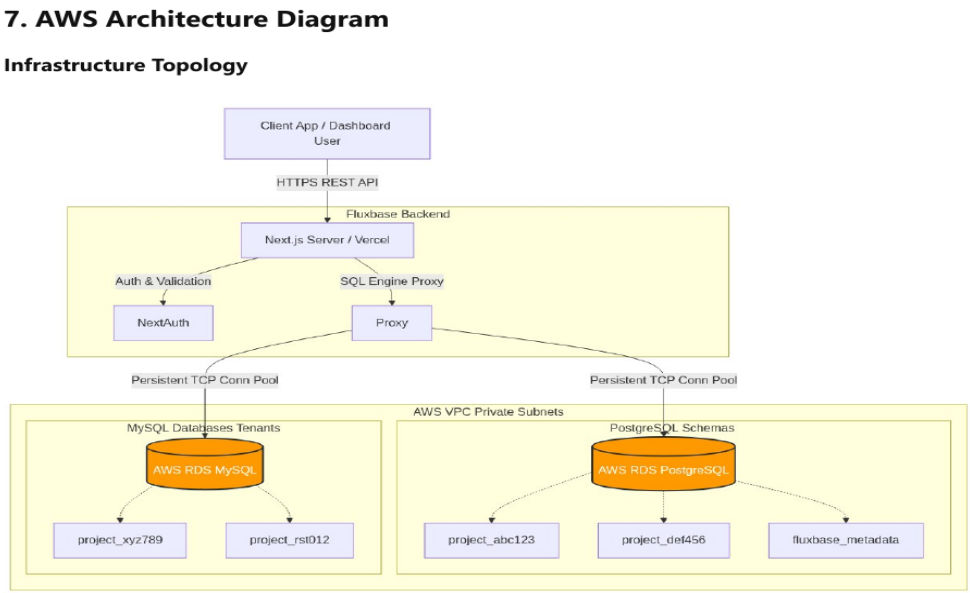

Figure 2 — AWS Infrastructure: Fluxbase Backend connecting to RDS MySQL & PostgreSQL via persistent TCP connection pools within a VPC

8. Production Hardening & Governance Layer

As Fluxbase evolves beyond early-stage prototype usage, a critical architectural enhancement is the introduction of a Governance & Protection Layer. While the current execution proxy securely isolates tenant schemas, additional controls are required to ensure predictable performance, cost protection, and multi-tenant fairness at scale.

8.1 Query Governance Engine

Fluxbase now enforces controlled SQL execution through the following safeguards:

Statement Whitelisting

Only the following SQL statements are permitted:

- `SELECT`
- `INSERT`
- `UPDATE`
- `DELETE`

Restricted operations:

- DROP DATABASE

- ALTER SYSTEM
- SET ROLE
- CREATE EXTENSION
- Cross-schema queries
- Administrative-level DDL

This prevents privilege escalation and infrastructure tampering.

Statement Timeout Enforcement

To prevent runaway queries and CPU exhaustion:

PostgreSQL:

```
SET statement_timeout = 5000;
```

MySQL:

```
SET SESSION max_execution_time = 5000;
```

All queries exceeding the defined threshold are automatically terminated.

Row Return Limits

To prevent large payload memory spikes:

- Maximum 1,000 rows per request
- Automatic LIMIT injection if not specified
- Server-side validation before execution

This ensures predictable memory and bandwidth usage.

9. Resource Isolation & Tenant Tiering

While Fluxbase currently operates on a single master AWS RDS instance, future growth requires workload isolation.

9.1 Noisy Neighbor Protection

Problem: Heavy analytical queries from one tenant can degrade performance for others.

Solution: Introduce workload routing tiers:

- **Shared Cluster:** Small and mid-tier tenants
- **Dedicated Cluster:** Enterprise / high-throughput tenants

Routing Logic: Enterprise tenants are dynamically routed to isolated RDS clusters to guarantee performance.

10. Connection & Performance Optimization

10.1 Persistent Connection Pooling

Fluxbase maintains aggressive TCP keep-alive connection pools to RDS using:

- pg (PostgreSQL)
- mysql2/promise (MySQL)

Enhancements:

- Integration with AWS RDS Proxy (optional)
- pgBouncer for large-scale Postgres workloads

This reduces login latency and eliminates cold-start connection overhead.

10.2 Read Replica Strategy

To improve read-heavy workloads:

- Primary RDS handles writes
- Read replicas handle analytical SELECT queries

This improves concurrency and reduces write contention.

11. Observability & Telemetry Layer

Production DBaaS platforms require deep insight into query behavior. Fluxbase introduces:

- Query latency histogram (p50, p95, p99)
- Rows returned tracking
- Per-tenant execution metrics
- Slow query logging

Example Telemetry Event:

```
{
  "tenant": "project_abc",
  "executionTimeMs": 124,
  "rowsReturned": 83,
  "dialect": "postgres",
  "cpuSnapshot": 14
}
```

This enables:

- Performance dashboards
- Abuse detection
- Capacity planning
- SLA enforcement

12. Cost & Abuse Protection

12.1 Rate Limiting

Per-tenant request rate limiting to prevent API abuse.

12.2 Circuit Breaker Pattern

If RDS CPU > 85% or connection pool > 90%:

- Heavy analytical queries are temporarily rejected
- Non-critical requests receive HTTP 429

- Core metadata operations remain protected

This prevents full platform collapse.

13. Caching Layer

To reduce repeated identical SELECT workloads:

- Redis-based query result caching
- Cache keyed by tenant + normalized SQL hash
- TTL-based invalidation

This dramatically reduces database load and improves perceived speed.

14. Scaling Strategy Roadmap

Phase 1 — Governance & Limits (Immediate)

- Statement timeout
- Row limits
- Rate limiting
- Query logging

Phase 2 — Performance Scaling

- Read replicas
- Query caching
- Tenant tier routing

Phase 3 — Enterprise Scaling

- Multi-cluster routing
- Dedicated RDS clusters
- Workload-aware sharding

End of Document